

Time Machine Project Proposal

Problem

Internet photo collections like MegaScenes contain over a century of images of the same places, including tourist snapshots, archival photographs, postcards, and often paintings and engravings that predate photography entirely. Almost all of the work on these collections tries to model variation in either viewpoint or appearance. Viewpoint gets handled by SfM, illumination and transients get absorbed by per-image appearance embeddings, and the fact that the scene itself was physically different in 1920 is ignored. We want to make time a first-class input. Our goal is to train a multi-view diffusion model that, given a few photos of a place and their dates, generates 3D-consistent views of that place at any other date.

A model like this could show what places with no old photo coverage looked like decades ago, reconstruct destroyed sites from a handful of archival photos plus what it has learned about similar buildings, give control over season, and (more speculatively) extrapolate into the future. It's also a way to ask how much history a generative model actually absorbs from dated images, which nobody has measured for 3D models. Prior temporal work on internet photos is per-scene reconstruction that can't transfer to new scenes or extrapolate past its data, and prior generative multi-view models are either atemporal or model time at the scale of seconds. As far as we can tell (we looked), no one has trained a feed-forward, cross-scene generative model conditioned on a calendar year.

Existing Literature

As of now, most existing work on modeling large outdoor scenes focuses on capturing variation in viewpoint and illumination within a single, fixed moment in time, as opposed to change across time itself. Datasets such as MegaScenes [1] have shown that large-scale internet photo collections can be used to fine-tune diffusion models for generalizable view synthesis. Building on this, Stable Virtual Camera (SEVA) [2] goes further with a generalist diffusion model that takes in any number of input views and generates 3D-consistent, temporally smooth views at arbitrary target cameras. In particular, it uses DUST3R [3] to estimate poses from unposed image collections, making it well suited for noisy datasets such as MegaScenes. At the same time, SEVA is purely spatial, since it has no mechanism for representing when an input or target image was captured. This is where our project comes in, as we aim to close the gap between generating new viewpoints of a scene and generating plausible views of how that same scene appeared, or might appear, at a different point in time.

The closest existing work on change over time is the chronology line of research on internet photos. 4D Cities [4] figured out when buildings existed from which features were visible in which historical photos, Scene Chronology [5] built space-time reconstructions from Flickr, Martin-Brualla and colleagues [6] mined multi-year 3D time-lapses out of internet photos, and

Neural Scene Chronology [7] fit a time-conditioned NeRF (with step functions for content that changes) on four landmarks. All of these are per-scene optimization, so they can replay the archive of one landmark but can't say anything about a scene they weren't fit on and can't extrapolate. On the generative side, 4DiM [8] conditions a diffusion model on both camera pose and time, but its time axis runs from seconds to seasons rather than decades, and WildCAT3D [9] handles MegaScenes-style appearance variation with no notion of when a photo was taken. A feed-forward model that actually knows what year it is looking at is the piece missing from all of the above.

Another existing model to consider is Instruct-GS2GS [10], which has demonstrated that diffusion-based edits to rendered views can be converted into a consistent 3D Gaussian Splatting [11] representation. While our primary approach relies on directly generating temporally-conditioned views with a fine-tuned SEVA model, Instruct-GS2GS remains an alternative for propagating temporal changes into 3DGS if needed.

Goals and Scope

The core question is whether a multi-view diffusion model can learn how scenes change over the years from dated wild photos, in a way that transfers to new scenes, and separately from how the photos themselves changed (film stock, grayscale, painting vs. photo). In practice that means adding per-image date conditioning to a pretrained multi-view diffusion model, fine-tuning it on a date-annotated subset of MegaScenes, checking how well it goes backwards on held-out archival photos and forwards on held-out recent ones, and then lifting the generated views into 3DGS so we get a navigable scene at whatever year we pick. If we have time, we can add other types of conditioning to our model, such as photo type (painting vs. grayscale vs. color film vs. modern digital photo) and month for seasonal changes. Painting-to-photo generation and probing what the model implicitly knows about history are other stretch goals. Most likely, our primary dataset for this project will be MegaScenes. MegaScenes is built from Wikimedia Commons and has roughly 430K scenes, around 100K SfM reconstructions, and something like 2M registered images, plus a much bigger pool of unregistered ones (which is where a lot of the old material lives, since SIFT doesn't match 1900 to 2020). The metadata probably doesn't have a clean date field, so we'll pull dates out of Commons categories, filenames, descriptions, EXIF, and Structured Data, and potentially store them as intervals or assign confidence values to them. We'll start with a few hundred big, date-diverse scenes and grow from there. The very first job is counting how many scenes actually have, say, 10+ registered pre-1970 images and 10+ modern ones, since that decides much of how ambitious the rest of this can be.

Our planned base model is Stable Virtual Camera (1.3B parameters, open weights, takes any number of input views and target cameras). For getting poses on old images we'll use MAST3R/DUST3R/VGGT against the existing COLMAP models, and for 3DGS we'll use gsplat. For baselines we'll compare against the unmodified base model (no date input), a nearest-dated-real-image retrieval baseline (probably surprisingly strong, since most scenes barely change), and Neural Scene Chronology on its own scenes if we can get it running.

In terms of compute, one of us has a Nexus account with access to A6000s and even H100/H200s. Most of our initial experiments will likely run as LoRA fine-tunes on A6000s. Once we get promising small-scale results, we can scale up.

Approach/Components

Our approach, at least the most basic version of it that we have in mind for initial experimentation, looks like this: (1) retrieve some of the big scenes from MegaScenes (bigger scene means more images, which means a higher likelihood of timestamp diversity), (2) process each image's metadata and extract a timestamp if available (sometimes this is in the image's name, description, or category), (3) create small training and validation datasets where each sample contains some input images, their camera intrinsics and extrinsics (or more likely the Plücker raymap), their timestamps, and the target images' camera data and timestamps (there will probably be about 5 input and target images per sample), (4) grab a pretrained open-source multi-view diffusion model, such as Stable Virtual Camera, and alter its conditioning so that it takes timestamps for each input and target image (as well as other potential modifications like type of photo, e.g. painting vs. grayscale vs. RGB), and (5) fine-tune our model on our data. Note: a lot of the details mentioned here, especially relating to our model's proposed architecture, are very likely to change. We have lots more ideas to try, such as embedding the timestamps into a higher-dimensional representation (this is a no-brainer), providing the month of each image to account for seasonality, and more. However, for our very first few experiments, what we'll be doing should be pretty close to what was described above. We will be using Python and PyTorch, building on the SEVA codebase and using the Diffusers library where it helps.

Documentation/Distribution

We will store all code in our GitHub repository. When writing code we will make sure to use comments where necessary to make the code more readable and easy for anyone to understand. We will have a README that gives an overview of the technical aspects of the code and where everything is located. This will also include steps for how anyone can clone the repo, install the needed dependencies, and run the different parts of the code (data pipeline, model fine-tuning, model sampling, and the sample viewer in the 3D environment).

Validation Methods

A big part of our project is building a pipeline that cleans our data and makes it usable for us to train on. For this we can use unit tests on test images to ensure the correct timestamp, orientation, and raw image (scaled to the desired resolution) are all parsed to our needed specifications. For the training process we can write unit tests to ensure that our diffusion model architecture properly takes in the input and propagates it as desired through the network (we can have test input with the correct dimensions and ensure we get an output of the correct dimensions). We can do the same for the feedforward 3DGS model, if we end up using one, to ensure the proper input gives us the desired output after going through the model. We can also

test the 3D environment visualization separately by giving it a test .ply file. During model training, we will validate on held-out views by calculating peak signal-to-noise ratio (PSNR) for how accurately we reconstruct them and LPIPS (Learned Perceptual Image Patch Similarity) for how perceptually close our output looks to the real image.

Deliverables

- Pipeline to gather data from sources and process it to fit our needs
- Modified SEVA architecture with timestamp conditioning (and potentially other architecture changes)
- Interactive 3D viewer demoing "scrub through time" on 2-3 example sites
- (Potential) Eval report comparing baseline SEVA to timestamp-conditioned outputs
- Final writeup / presentation

Why an agent can't do this

This specific idea is mostly new, so it would require a lot of experimentation that isn't feasible for an agent to do all by itself. As our project involves 3D Gaussian Splatting, which works by running gradient descent over millions of parameters to minimize pixel error, a big chunk of the work is an optimization loop that has to be run on a GPU and whose results have to be looked at, which isn't something an LLM can reason its way through. Even more so with diffusion models, which generate pixels for unseen viewpoints because they've learned a visual prior from massive image datasets. That's a statistical mapping rather than a series of logical steps, which goes beyond what a standard agent can do. An agent can write the parser, the dataloader, and the training loop, but the tasks we expect to need human effort and intuition are: deciding whether a Commons date is a capture date or a scan date, picking test scenes with known changes, judging whether a generated 1920 view is plausible or confidently made up, deciding what an ambiguous result means and which experiment deserves our limited GPU time, and deciding what to release, given that this is a tool for making convincing images of real places at dates that never existed.

Timeline

Phase 1a (1-2 weeks): Data pipeline to process and clean the MegaScenes dataset with image, timestamp, and camera orientation. The goal is to have a PyTorch dataset of features ready for training. Figure out which subset of the data we actually want to download (since there is a lot available).

Phase 1b (1-2 weeks), in parallel with 1a: Experiment with 3DGS models (feedforward like AnySplat, or an optimization-based one). Finalize one, or set up the pipeline so the user can choose.

Phase 2a (1-2 weeks): Modify the SEVA architecture to be conditioned on timestamps.

Phase 2b (1-2 weeks), in parallel with 2a: Use libraries like gsplat or SuperSplat to visualize the 3D environment from the 3DGS output. Finalize this visualization pipeline.

Phase 3 (3-4 weeks): Fine-tune the diffusion model and evaluate it with the metrics outlined above. Use LoRA to be more compute-efficient. Make any refinements to our overall methodology as needed and keep improving it.

Phase 4 (1 week): Construct the final presentation and writeup outlining the entire process.

Repo

<https://github.com/varunm18/diffusion-time-machine>

References

1. J. Tung et al. MegaScenes: Scene-Level View Synthesis at Scale. ECCV 2024.
<https://megascenes.github.io/>
2. J. Zhou et al. Stable Virtual Camera: Generative View Synthesis with Diffusion Models. ICCV 2025. <https://github.com/Stability-AI/stable-virtual-camera>
3. S. Wang et al. DUST3R: Geometric 3D Vision Made Easy. CVPR 2024; V. Leroy, Y. Cabon, J. Revaud. Grounding Image Matching in 3D with MAST3R. ECCV 2024.
<https://github.com/naver/mast3r>
4. G. Schindler, F. Dellaert, S. B. Kang. Inferring Temporal Order of Images from 3D Structure. CVPR 2007.
5. K. Matzen, N. Snavely. Scene Chronology. ECCV 2014.
6. R. Martin-Brualla, D. Gallup, S. M. Seitz. 3D Time-lapse Reconstruction from Internet Photos. ICCV 2015.
7. H. Lin, Q. Wang, R. Cai, S. Peng, H. Averbuch-Elor, X. Zhou, N. Snavely. Neural Scene Chronology. CVPR 2023.
8. D. Watson et al. Controlling Space and Time with Diffusion Models (4DiM). arXiv 2024.
9. M. Alper, D. Novotny, F. Kokkinos, H. Averbuch-Elor, T. Monnier. WildCAT3D: Appearance-Aware Multi-View Diffusion in the Wild. arXiv 2025.
<https://arxiv.org/abs/2506.13030>
10. C. Vachha, A. Haque. Instruct-GS2GS: Editing 3D Gaussian Splats with Instructions. 2024.
<https://github.com/cvachha/instruct-gs2gs>
11. B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. SIGGRAPH 2023.
<https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>